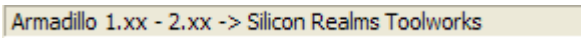
Programa:	Metal Wax Casting Converter DEMO 3.0	
Protección:	Armadillo 2.85 + CopyMemII	
Objetivos:	Desempakar.	
URL descarga:	www.color-picker.net	
Dificultad:	Fácil si habéis leído las teorías	
Herramientas:	OllyDbg + PEiD + LordPE + Import Reconstructor.	
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos]	Fecha: 25-08-2005

Hola a todos otra vez..!!

Hoy entraremos de pleno en los mundos de ***Armadillo*** y su hiper-conocido ***CopyMemII***, esta es una protección que asusta la primera vez que te la encuentras, recuerdo mi primer encontronazo con un Armadillo de este tipo en el que no sabía como sacarle el ***OEP*** y me quedaba dando vueltas en el loop del padre entre ***WaitForDebug***… ***ContinueDebug***…, etc.., era espantoso pues no sabía absolutamente ***NADA*** de lo que estaba sucediendo, es más ni si quiera me imaginaba lo que pasaba, fue entonces cuando sentí la enorme necesidad de documentarme en este tema y ahí descubrí lo que realmente hace ***Armadillo***, recomiendo encarecidamente leer las teorías de los compañeros de ***[CracksLatinos]*** porque son directamente buenísimas, no dejan lugar a dudas, desde el ***Armadillo*** más simple como el que veremos ahora hasta el más complejo con ***Nanomites***, están absolutamente todos.

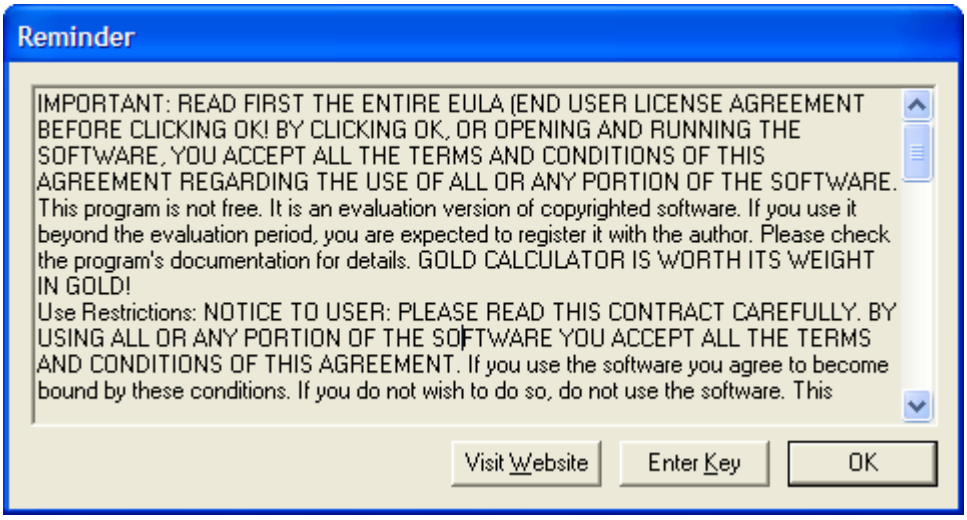
Dicho esto passemos a la carga con lo que nos trae hoy, primero de todo abriremos el ejecutable desde ***PEiD*** o ***RDG Packer*** lo mismo da, en este caso el segundo es mejor ya que nos precisa la versión exacta:

PEiD:

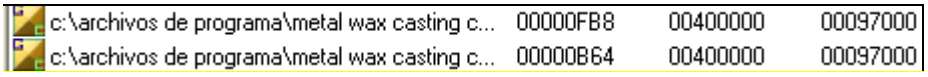


RDG Packer:

Ambos nos dicen que ***Armadillo*** es el rival pero el segundo nos informa que se trata de la vresión ***2.85***, una de las antiguas de este protector teniendo en cuenta que ya han superado la versión ***4***, antes de ir al a ataque deberemos precisar cierta información, miraremos cuantas instancias se crean del proceso, ejecuto el archivo y aparece esta nag:



Es la que muestran muchos programas protegidos con ***Armadillo***, parece que es la estandar que pone, por lo visto le puedes decir que te ponga una ***nag*** que será esta supongo, vale pulsamos ***OK,*** abrimos ***LordPE*** y miramos en la lista de procesos:



Se ve claro que son dos los procesos, uno actúa como padre y otro como hijo, el padre como todos sabemos está depurando al hijo así que no podemos depurarlo nosotros mientras que el padre lo haga, deberíamos primero liberarlo de esa pesada carga ;-) si nos adelantamos y eliminamos el padre de inicio mediante la famosa ***API OpenMutexA*** cuando nuestor hijo intente saltar al ***OEP*** ¿quién le desencriptará el código si tiene ***CopyMem***? Y si tiene Nanomites quién los procesará? Estos son los dos grandes inconvenieentes a mi modo de ver las cosas,, ahora que sabemos que el padre está deberemos conocer su grado de implicación, si está por motivos de ***Nanomites*** (los ***INT3*** que pone en la sección de código) o bien por ***CopyMem***, esto tiene fácil solución, carguemos el programa en ***Olly*** y pongamos un ***BP*** en ***WriteProcessMemory*** y veamos si le copia bloques de ***1000h*** bytes o no:

Como no se ha quejado es que el ***BP*** está bien puesto, ahora damos ***RUN*** y cuando pare miremos en el stack, decir que parará dos veces antes de la ***nag***, a estas de momento no les hagamos mucho caso, pero en la tercera parada vemos esto:

```
0012E1D8  0042F4A4  CALL to WriteProcessMemory from CASCAL~D.0042F49E
0012E1DC  0000004C  hProcess = 0000004C (window)
0012E1E0  00401000  Address = 401000
0012E1E4  00392630  Buffer = 00392630
0012E1E8  00001000  BytesToWrite = 1000 (4096.)
0012E1EC  0012E2EC  pBytesWritten = 0012E2EC
```

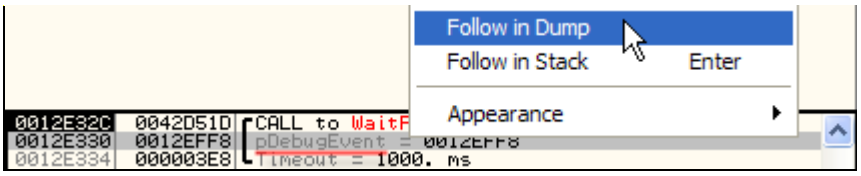
Vale tiene ***CopyMemII***, todos sabemos que el padre le irá copiando trozos de código de ***1000h*** byes al hijo tal y como los vaya necesitando y para evitar el dumpeo cuando ya los ha utilizado los va borrando de la memoria, de modo que el archivo no llega a estar nunca completo en memoria, pero todo truco tiene su solución como pronto veremos, si tiene o no ***Nanomites*** ya lo veremos después esto no es problema, lo primero es lo primero, evitemos el ***CopyMem*** antes.

Venciendo el CopyMemII…

A ello vamos, hay una serie de pasos descritos en las teorías para vencer este método, las utilizaremos tambieén, la podéis encontrar también en el anterior tut de ***ARWizard 3.1*** bien descritos allí, en este caso y como es todo bien conocido y documentado nos detendremos en algunas particularidades si las hay, el resto lo enumeraremos de pasada, repito para profundizar más ya sabéis donde acudir (me podéis pedir ayuda también si no os sale algo o no entendéis algo de lo que aquí se explica para esos estamos…)

Haremos un rápido resumen del método aplicado a este caso concreto, lo primero de todo es reiniciar, pulsamos Ctrl+F2 y quedaremos detenidos en el Entry Point, seguidamente haremos… doy por hecho que el debugger lo tenemos oculto mediante el plugin IsDebuggerPresent, sino podemos hacerlo a mano, cambiando el byte de ***[7FFDF002]=0*** y lisos.. ahora si:

1.- BP en WaitForDebugEvent: Esto es importante porque del reporte obtendremos el ***OEP*** ya que el hijo debe especificar el tipo de evento que sucedió y donde (el contexto) y entre estos datos irá el ***OEP***, jeje.. ponemos el ***BP*** y damos ***RUN*** al poco ***Olly*** parará, en estos momentos aún no tenemos nada ya que el hijo aún no ha intentado saltar al ***OEP***, antes debemos pasar por la nag y todo eso, bien vamos al stack y en el valor del parámetro ***pDebugEvent*** hacemos un ***Follow in Dump*** con el botón derecho, esto nos dejará el ***reporte*** a la vista:



2.- Quitar el anterior BP de WaitForDebugEvent: Esto se hace para que quede el reporte de la ***API*** a la vista, sino sucesivas llamadas a esta nos lo borrarían y nos va muy bien tenerlo, entre otras cosas porque veremos el ***OEP*** allí y aparte para el injerto que luego haremos lo necesitaremos también.

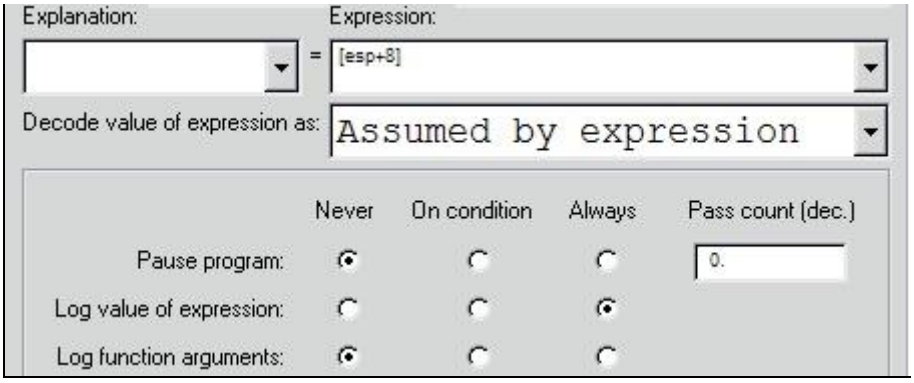
3.- Poner un BP en la api WriteProcessMemory: Vale ponemos ahora un ***BP*** en esta ***API*** y damos ***RUN***, saldrá la nag en pantalla, aceptamos picando en "***OK***" y ***Olly*** se detiene donde queríamos, si miramos en el reporte veremos el ***OEP***:

```
Address   Hex dump                                          ASCII
0012EFF8  01 00 00 00 90 09 00 00 54 06 00 00 01 00 00 80  0...É...T♠..0..Ç
0012F008  00 00 00 00 00 00 00 00 08 1D 40 00 02 00 00 00  ........◘#@.☻...
0012F018  00 00 00 00 08 1D 40 00 08 1D 40 00 00 00 00 00  ....◘#@.◘#@.....
0012F028  01 E4 58 E1 00 00 00 00 18 6D D7 F4 00 00 00 00  ☺ä×ß....↑mï¶....
0012F038  13 00 00 00 38 2D 50 00 64 6D D7 F4 5E CC 4D 80  !!...8-P.dmï¶^╠MÇ
0012F048  00 00 00 00 08 1D 40 00 01 00 00 00 01 00 00 00  ....◘#@.☺...☺...
0012F058  01 00 6E 00 38 00 00 00 D7 1A EE 05 FF FF FF FF  ☺.n.8...ï→^♣
```

En la imagen se ve en tres posiciones distintas pero en realidad aparece en seis lo que las otras están un poco más arriba por eso no se ven en la imagen, se ve claramente que el está en ***:401D08***, fijáos la importancia del reporte y los datos que nos va dando. Lo anotamos po rupuresto:

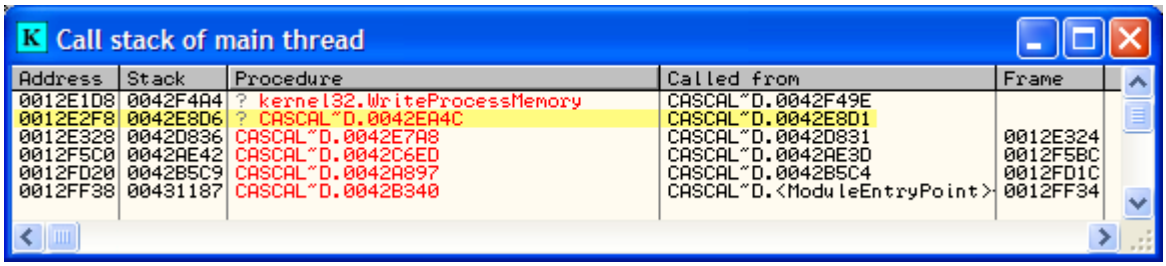
OEP: 401D08

4.- Quitar el Bp de WriteProcessMemory y ponerlo Conditional Log: Lo configuramos así:



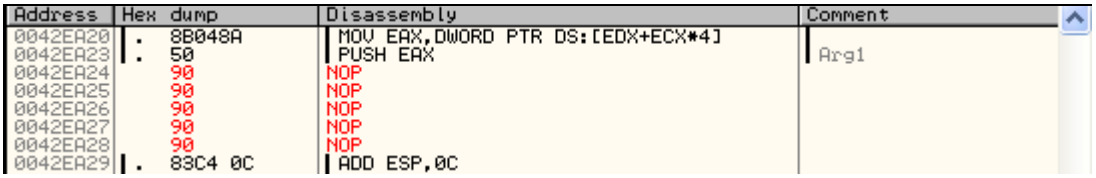
Esto hará que todos los bloques que se copien en el hijo queden "***loggeados***" en el pertinente informe, además picamos en el botón de los puntos suspensivos y elegimos "***Log to file***" para que lo guarte a un archivo ***.txt*** cuyo nombre nos pedirá, esta información nos será útil por si algo falla ver que de esto no ha sido.

5.- Nopeo del call destructor de bloques: Ahora buscaremos el otro call que va a ***WriteProcessMemory*** que se encarga de destruir los bloques que ya han sido utilizados por el programa, esto es parte de la protección de ***Armadillo***. Es muy fácil buscar este call porque apunta al mismo lugar que e call en el que nos encontramos ahora, nos podemos apoyar si queremos en el ***Call Stack*** aunque no es obligatorio, veamos que nos dice:

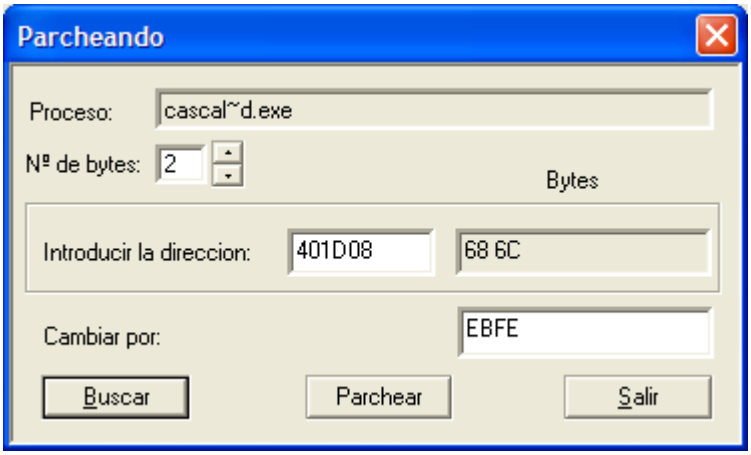


El call que he seleccionado nos servirá de guía, picamos con el botón derecho y elegimos ***Show Call*** y caemos como bien marca en su parte derecha en ***:42E8D1*** y es un ***CALL*** (donde estamos ahora) picamos enter para entrar y caemos en su primera instrucción que es ***PUSH EBP***, vale busquemos cuantas referencias hay a este con ***Ctrl+R***, aparecerán dos, la nuestra donde estamos ahora y una segunda, lógicamente la que debemos nopear.

<u>Call destructor</u> en ***:42EA24*** => Lo nopeamos



7.- Crear un loop infinito en el hijo: Para esto utilizamos ***Pupe***, conociendo el ***OEP*** no es difícil hacerlo, antes debemos dejar que se ejecute la ***API WriteProcessMemory*** en la que estamos detenidos para que precisamente copie el bloque del ***OEP*** sino ***Pupe*** nos dará un error y con razón, o sea picamos ***Ctrl+F9*** (ret) y ahora sí lo hacemos:



Como se puede apreciar es una aplicación en ***Visual Basic*** por eso no vemos el típico ***558B*** (***PUSH EBP***…) y en cambio vemos ***686C*** (***68*** es el ***OPCODE*** de ***PUSH***) y como sabemos que los progrmaas compilados en ***Visual*** comienzan así seguido de una llamada a ***ThunRTMain*** que ejecuta el programa, pulsamos ***Parchear*** y seguimos…

8.- Nopeo de WaitForDebugEvent: Ahora debemos nopear esta ***API*** ya que el injerto que haremos así lo requiere, ponemos un ***BP*** en ella y cuando ***Olly*** pare vamos al stack para ver el lugar de la llamada:

```
0012E32C  0042D51D  CALL to WaitForDebugEvent
0012E330  0012EFF8  pDebugEvent = 0012EFF8
0012E334  000003E8  Timeout = 1000. ms
0012E338  0012FF2C
```

Vamos hacia ***:42D51D*** y nos cargamos de raíz la llamada a la ***API***, no olvidar el nopeo de los ***PUSH*** que lleva consigo que son dos (***PUSH 3E8*** y ***PUSH ECX***) en definitiva, se nopeará lo que vemos aquí seleccionado:

```
Address   Hex dump   Disassembly                          Comment
0042D505  .v 0F84 5212000( JE CASCAL~D.0042E75D
0042D50B     68 E8030000   PUSH 3E8                       Timeout = 1000. ms
0042D510     8B8D 38FAFFF( MOV ECX,DWORD PTR SS:[EBP-5C8]
0042D516     51            PUSH ECX                       pDebugEvent
0042D517     FF15 8C60440( CALL DWORD PTR DS:[<&KERNEL32.WaitForD  WaitForDebugEvent
0042D51D  .  85C0          TEST EAX,EAX
0042D51F  v  0F84 3312000( JE CASCAL~D.0042E758
0042D525  .  33D2          XOR EDX,EDX
```

He aprovechado la imagen para marcar en ***ROJO*** el salto que dirigiremos hacia nuestra rutina cuya misión el lograr que el padre copie todos los bloques en memoria, uno detrás del otro y como ya hemos neutralizado el call destructor estarán ahí a la espera de ser dumpeados, jajaja..

9.- Realización del injerto: Este es un paso especial podríamos decir, de su correcto funcionamiento depende el éxito de toda la operación, como antes comenté la información necesaria para este la encontramos en el reporte que quedó de la primera llamada a ***WaitForDebugEvent***, necesitamos las posiciones en las que aparece el ***OEP***, he leído muchos tuts en los quesólo tienen en cuenta tres posiciones de estas pero para mayor seguridad yo siempre las cojo todas, veámoslas:

```
Address   Hex dump                                         ASCII
0012EFB8  00 00 14 00 00 00 00 00 00 00 14 00 08 06 13 00  ..¶.......¶.◘♠‼.
0012EFC8  00 00 00 00 D0 3B FB 77 48 ED 12 00 00 00 00 00  ....ŝ;'wHŷ♯.....
0012EFD8  7C F2 12 01 54 ED 12 00 08 1D 40 00 08 1D 40 00  |=♯0Tŷ♯.◘#@.◘#@.
0012EFE8  00 00 00 00 01 D8 F5 77 01 00 01 80 F8 EF 12 00  ....0ïŝw0.0Ç°'♯.
0012EFF8  01 00 00 00 0C 0F 00 00 E0 0E 00 00 01 00 00 80  0....*..óß..0..Ç
0012F008  00 00 00 00 00 00 00 00 08 1D 40 00 02 00 00 00  ........◘#@.☻...
0012F018  00 00 00 00 08 1D 40 00 08 1D 40 00 00 00 00 00  ....◘#@.◘#@.....
0012F028  01 E4 58 E1 00 00 00 00 18 7D 23 F5 00 00 00 00  0δXß....↑}#§....
0012F038  13 00 00 00 38 2D 50 00 64 7D 23 F5 5E CC 4D 80  ‼...8-P.d}#§^╠MÇ
0012F048  00 00 00 00 08 1D 40 00 01 00 00 00 01 00 00 00  ....◘#@.0...0...
0012F058  01 00 6E 00 38 00 00 00 14 00 2B 04 FF FF FF FF  0.n.8...¶.+◆
```

Estos son los lugares en mi máquina, por su puesto que en la tuya cambiarán , ya contamos con esto, lo importante sobretodo es no equivocarse a la hora de hacer referencia a ellas y antes de empezar debemos poner en los ***6*** lugares el valor ***400000*** para que el ***injerto*** funcione bien, quedará pues así:

```
Address   Hex dump                                         ASCII
0012EFB8  00 00 14 00 00 00 00 00 00 00 14 00 08 06 13 00  ..¶.......¶.◘♠‼.
0012EFC8  00 00 00 00 D0 3B FB 77 48 ED 12 00 00 00 00 00  ....ŝ;'wHŷ♯.....
0012EFD8  7C F2 12 01 54 ED 12 00 00 00 40 00 00 00 40 00  |=♯0Tŷ♯...@...@.
0012EFE8  00 00 00 00 01 D8 F5 77 01 00 01 80 F8 EF 12 00  ....0ïŝw0.0Ç°'♯.
0012EFF8  01 00 00 00 0C 0F 00 00 E0 0E 00 00 01 00 00 80  0....*..óß..0..Ç
0012F008  00 00 00 00 00 00 00 00 00 00 40 00 02 00 00 00  ..........@.☻...
0012F018  00 00 00 00 00 00 40 00 00 00 40 00 00 00 00 00  ......@...@.....
0012F028  01 E4 58 E1 00 00 00 00 18 7D 23 F5 00 00 00 00  0δXß....↑}#§....
0012F038  13 00 00 00 38 2D 50 00 64 7D 23 F5 5E CC 4D 80  ‼...8-P.d}#§^╠MÇ
0012F048  00 00 00 00 00 00 40 00 01 00 00 00 01 00 00 00  ......@.0...0...
0012F058  01 00 6E 00 38 00 00 00 14 00 2B 04 FF FF FF FF  0.n.8...¶.+◆
```

Si no hacemos esto de poco servirá el trabajo que hagamos, y repito que es muuuuuuuuuuuuuy importante no errar en esta parte ya que estamos casi al final del trabajo y repetirlo todo, bouuuuuuff… no queremos no.. entonces prestaremos mucha atención y acabaremos pronto. El injerto lo pondremos como viene siendo habitual a partir de ***:401000***, a la salida del ***call WaitForDebugEvent*** hay un salto que reemplazaremos por uno hacia nuestro injerto:

```
Address   Hex dump   Disassembly                          Comment
0042D51B     90       NOP
0042D51C     90       NOP
0042D51D  .  85C0     TEST EAX,EAX
0042D51F  - E9 DC3AFDFF JMP CASCAL~D.00401000
0042D524     90       NOP
0042D525  .  33D2     XOR EDX,EDX
```

Se ven restos del ***CALL*** nopeado y acto seguido el salto al lugar donde colocaremos la rutina que nos haga el trabajito sucio, la misión de esta o el objetivo como lo queramos llamar está explicado antes, para refrescar la memoria es el de obligar al proceso ***PADRE*** a escribir todos los bloques en la memoria y así poder obtener un dumpeado del archivo, si no hacemos esto lógicamente nos faltaría código para hacerlo funcionar.

Antes de seguir deberemos mirar el tamaño de la primera sección (la de código donde ***Armadillo*** hace de las duyas) para ponerle una condición de salida al bucle en el que vamos a meter el program, veamos:

00400000	00001000	CASCAL~D		PE header	Imag	R	RWE
00401000	00022000	CASCAL~D	.text		Imag	R	RWE
00423000	00003000	CASCAL~D	.data		Imag	R	RWE
00426000	00020000	CASCAL~D	.text1	code	Imag	R	RWE
00446000	00020000	CASCAL~D	.data1	data,import	Imag	R	RWE
00466000	00030000	CASCAL~D	.pdata		Imag	R	RWE
00496000	00001000	CASCAL~D	.rsrc	resources	Imag	R	RWE

La que está marcada es la que a nosotros nos interesa, su inicio está en ***:401000*** como siempre y el tamaño es ***22000h***, entonces su final está en ***401000+22000=423000***, esta es la condición de salida entonces, ahora sí podemos injertar:

Address	Hex dump	Disassembly	Comment
00401000	8105 E0EF1200 00100000	ADD DWORD PTR DS:[12EFE0],1000	
0040100A	8105 E4EF1200 00100000	ADD DWORD PTR DS:[12EFE4],1000	
00401014	8105 10F01200 00100000	ADD DWORD PTR DS:[12F010],1000	
0040101E	8105 1CF01200 00100000	ADD DWORD PTR DS:[12F01C],1000	
00401028	8105 20F01200 00100000	ADD DWORD PTR DS:[12F020],1000	
00401032	8105 4CF01200 00100000	ADD DWORD PTR DS:[12F04C],1000	
0040103C	813D E0EF1200 00304200	CMP DWORD PTR DS:[12EFE0],CASCAL~D.00423000	
00401046	74 05	JE SHORT CASCAL~D.0040104D	
00401048	E9 D8C40200	JMP CASCAL~D.0042D525	
0040104D	90	NOP	
0040104E	0000	ADD BYTE PTR DS:[EAX],AL	
00401050	0000	ADD BYTE PTR DS:[EAX],AL	

Vemos como la condición de salida es que el bloque ***[12EFE0]*** alcance la cifra ***423000*** que es el final de la seción tratada, cuando termine el trabajito quedará parado en el ***INT3*** (***BPX*** que pusimos) y así nos retornará el control para que finalicemos el trabajo, es importante antes de seguir ir a la ventana donde salvaremos todo en un ***Log*** (archivo de texto) picando el ***Log to File*** y poniéndole un nombre para luego ver que realmente los bloques se copiaron todos, a esta ventana se accede pulsando en el botón de los puntos suspuensivos. También es conveniente hacer una pasada al injerto sin dar ***RUN*** todavía para asegurarse de que funciona bien y no fallará, para empezar con ***New Origin Here*** situamos el ***EIP*** en ***:401000*** y empezamos

Los bytes del injerto son:

81 05 E0 EF 12 00 00 10 00 00 81 05 E4 EF 12 00 00 10 00 00 81 05 10 F0 12 00 00 10 00 00 81 05
1C F0 12 00 00 10 00 00 81 05 20 F0 12 00 00 10 00 00 81 05 4C F0 12 00 00 10 00 00 81 3D E0 EF
12 00 00 30 42 00 74 05 E9 D8 C4 02 00 90

Así no lo tendréis que copiar, simplemente con ***Ctrl+C*** os lleváis los bytes a ***Olly*** que con ***Binary Paste*** los pegará bien. Estábamos en el injerto, damos ***RUN*** y cuando ***Olly*** pare miramos en la barra de estado:

Breakpoint at CASCAL~D.0040104D

Está claro que fue por el ***BP***, no ocurrió ninguna excepción ni nada raro, parece que vamos bien entonces, si miramos en el log vemos que los bloques efectivamente se copiaron todos, o sea que el ***CopyMem*** a estas alturas está vencido, nos queda separar el padre del hijo y dumpear, separémoslos primero, vamos al ***Pupe*** y miremos el ***PID*** del ***hijo*** que es de los dos el de arriba, en este caso vale ***C7C*** (este es el ***handle***), después de nuesro injerto ponemos:

Address	Hex dump	Disassembly	Comment
0040104D	90	NOP	
0040104E	68 7C0C0000	PUSH 0C7C	
00401053	E8 DBE1A877	CALL kernel32.DebugActiveProcessStop	
00401058	90	NOP	
00401059	90	NOP	

Damos ***F9*** y cuando para en el ***BP*** si tenemos ***EAX==1*** hemos ganado, sino repetir la jugada revisando el ***PID*** del hijo que es el que se debe ***PUSHEAR***. El siguiente paso será abrir otro ***Olly*** para atachear al hijo ya que no está siendo depurado por nadie y está accesible a ***Olly***, hay quien dice que no cerremos este ***Olly*** porque sino el hijo se comportará como padre y creará otro proceso, la verdad que quizá tienen razón aunque a veces lo he cerrado y no ocurrión nada ¿?

Atacheado el hijo queda así:

Attached process paused at ntdll.DbgBreakPoint

Esto es buena señal, ahora damos ***RUN*** y Pause (***F12***) para quedar en el ***OEP*** del programa, recordad el loop infinito que le hicimos con ***Pupe***, ahora nos permite plantarnos en el ***OEP*** del programa sin grandes dificultades, restauramos los bytes originales ***686C*** en lugar del ***EBFE*** para dejar todo como estaba:

Address	Hex dump	Disassembly	Comment
00401D08	68 6C564000	PUSH CASCAL~D.0040566C	
00401D0D	E8 F0FFFFFF	CALL CASCAL~D.00401D02	JMP to MSVBVM60.ThunRTMain
00401D12	0000	ADD BYTE PTR DS:[EAX],AL	
00401D14	0000	ADD BYTE PTR DS:[EAX],AL	

Vale busquemos la ***IAT***, como estamos en ***Visual Basic*** sólo debemos subir un poco con la ruedecilla del ratón para llegar a visualizar la tabla de saltos, subimos un poco y enseguida encontramos el inicio de esta:

```
Address   Hex dump                                          ASCII
00401000  B9 F9 0D 66 06 88 0F 66 79 FE 0E 66 E9 F9 0D 66  ¶¨.f♠ê*fy■ßfÚ¨.f
00401010  64 C1 0C 66 4C EA 0F 66 B4 8B 10 66 DB 0D 0E 66  d┴.fLÚ*f┤ï►f■.ßf
00401020  5F 5F 0E 66 A2 72 10 66 3A F7 0E 66 3B 9A 0D 66  __ßfór►f:·ßf;Ü.f
00401030  7B 65 0E 66 C1 FD 0E 66 3A 5F 0E 66 5A C2 0C 66  {eßf┴²ßf:_ßfZ┬.f
```

Se ve fácil porque coincide con el inicio de la sección .text, busquemos el final:

```
Address   Hex dump                                          ASCII
00401130  C8 57 0F 66 6D E3 0E 66 84 8B 10 66 C4 8A 0F 66  ╚W*fmÐßfäï►f─è*f
00401140  91 D1 0E 66 27 9A 0D 66 B0 60 0E 66 D4 2D 0D 66  æÐßf'Ü.f▒'ßfÈ-.f
00401150  00 00 00 00 00 00 00 00 01 00 08 00 56 2B 41 00  ........☺.◘.V+A.
00401160  05 00 08 00 1D 2C 41 00 00 00 00 00 24 2C 41 00  ♣.◘.#,A.....$,A.
```

Este es el final, su longitud se ogtiene rápidamente con una simple resta:

<u>Longitud IAT</u> = Final – Inicio: 401150 – 401000 = **150**

Como se ve es una tabla pequeñita, jejeje… venga dumpeemos, si no queremos tener problemas debemos reparar la cabecera que ***Armadillo*** nos ha dañado, en concreto nos ha desviado la signature que no más que un puntero a la cadena ***PE*** y también nos ha modificado el número de sercciones. Por partes, para el primer caso nos desplazamos al ***offset 0x3Ch*** (nunca cambia por ***standard***) y cambiamos el valor allí presente por el ***offset*** al ***PE***, veamos donde está éste:

```
Address   Hex dump                                          ASCII
004000C8  50 45 00 00 4C 01 08 00 6B E8 7D 3E 00 00 00 00  PE..L☺◘.kþ}>....
004000D8  00 00 00 00 E0 00 0F 01 0B 01 53 52 00 10 01 00  ....α.*☺♂☺SR.►☺.
004000E8  00 50 03 00 00 00 00 00 CD 10 03 00 00 60 02 00  .P♥.....=►♥..'☻.
```

Vale lo tenemos en ***:4000C8***, su offset será ***4000C8 – 400000 = 0C8h***, vamos al signature y lo cambiamos:

```
Address   Hex dump                                          ASCII
0040003C  2D 2C 08 00 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C  -,◘.ßº╢ß.┤.=↑☺☺L
0040004C  CD 21 54 68 69 73 20 70 72 6F 67 72 61 6D 20 63  =↑This program c
0040005C  61 6E 6E 6F 74 20 62 65 20 72 75 6E 20 69 6E 20  annot be run in
```

Mirad como nos lo había modificado malintencionadamente, creedme que si no hacemos esto ahora nos dará problemas para reparar la ***IAT*** con le Import, ponemos el valor del ***offset*** al ***PE*** como hemos quedado:

```
Address   Hex dump                                          ASCII
0040003C  C8 00 00 00 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C  ╚...ßº╢ß.┤.=↑☺☺L
0040004C  CD 21 54 68 69 73 20 70 72 6F 67 72 61 6D 20 63  =↑This program c
0040005C  61 6E 6E 6F 74 20 62 65 20 72 75 6E 20 69 6E 20  annot be run in
```

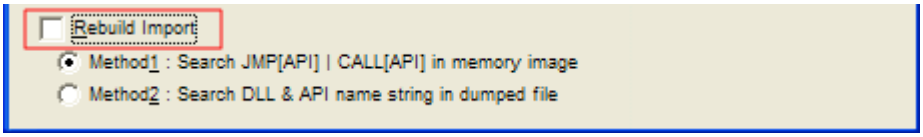
Con esto solventamos el primer problema, nos queda el segundo, al número de secciones del archivo, si vamos a la ventana de memoria con ***Alt+M*** veremos que vuelve a aparecer la cadena ***PE header*** en la cabecera (ya la reconoce) hacemos doble clic sobre ella y se abrrá en una nueva ventana, si no se ve digamos en modo texto picamos con el botón derecho en la opción ***Special->PE*** header que nos lo mostrará todo perfecto, bajamos un poco hasta aquí:

```
004000C8   50 45 00 0(  ASCII "PE"        PE signature (PE)
004000CC   4C01         DW 014C            Machine = IMAGE_FILE_MACHINE_I386
004000CE   0800         DW 0008             NumberOfSections = 8
004000D0   6BE87D3E     DD 3E7DE86B         TimeDateStamp = 3E7DE86B
004000D4   00000000     DD 00000000         PointerToSymbolTable = 0
```

Podemos mirar el archivo con ***PEditor*** y ver que realmente tiene sólo seis y no ocho, esto no estoy del todo seguro los problemas que nos puede acarrear pero por si las moscas lo repararemos está claro, picamos con el botón derecho en el valor malo y elegimos ***Modify Integer***, se abrirá una pequeña ventanita donde cambiamos el ***8*** opr un ***6*** y listos:

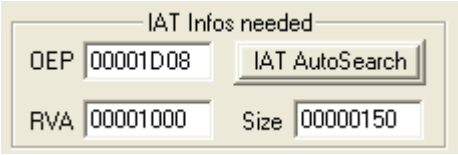
```
004000C8   50 45 00 0(  ASCII "PE"        PE signature (PE)
004000CC   4C01         DW 014C            Machine = IMAGE_FILE_MACHINE_I386
004000CE   0600         DW 0006             NumberOfSections = 6
004000D0   6BE87D3E     DD 3E7DE86B         TimeDateStamp = 3E7DE86B
004000D4   00000000     DD 00000000         PointerToSymbolTable = 0
```

Y por fin ya podemos dumpear con total normalidad como si nada, yo lo haré con el mismo ***Ollydump***, también podría utilizar ***LordPE*** que va muy bien, como siempre destildo la opción ***Rebuid Import***:

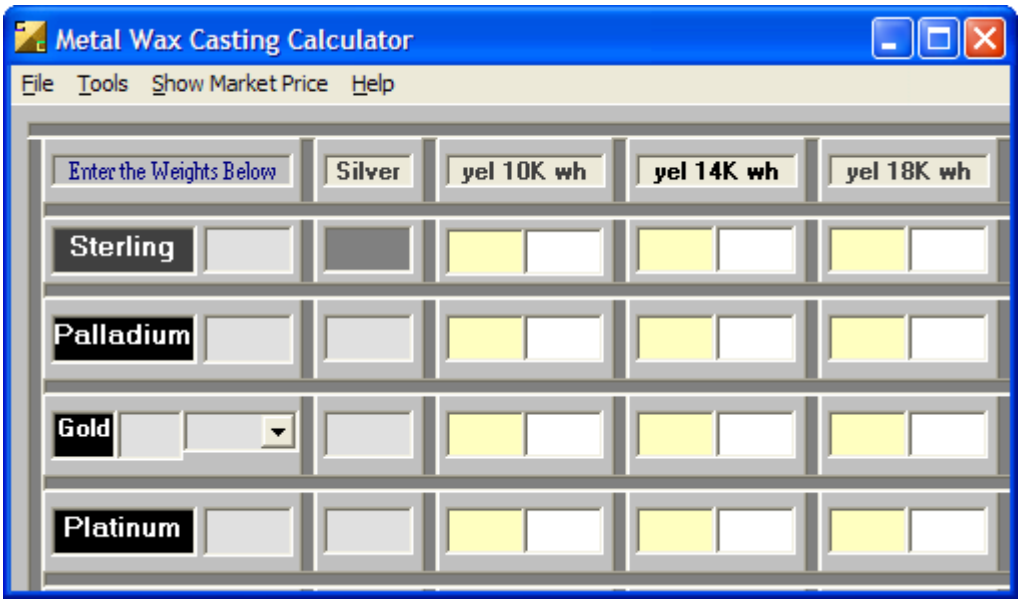


Esto lo hacemos así ya que será con el ***Import Reconstructor*** con el que vamos a repararle la ***IAT***, además en estos casos ***Olly*** poco puede hacer, probadlo y luego me comentáis cuando os empiece a tirar errores el dumpeado.

Y por fin le llegó el turno al ***Import***, como conocemos el ***OEP***, el inicio de tabla y su largo podemos darle los valores nosotros mismos evitando el ***IAT Autosearch*** que a veces va y otras no va, le daremos estos datos (recordad que trabaja con ***offsets*** y no con direcciones de memoria, si el ***OEP=401D08*** se le dará ***OEP=1D08***) venga la ponemos esto:



Luego picamos en ***Get Imports*** para que lea las direcciones de la misma tabla y nos mostrará un solo ***Thunk*** del tipo ***msvbvm60.dll*** del todo correcto, o sea qu e nos ahorramos buscar el mágico, pues vale ***Fix Dump*** y elegimos el dump en el pequeño explorer que se abre, nos dejará el archivo reparado llamado ***dump_.exe*** que si probamos…



Sale corriendo como era de esperar, se nota que es una versión algo antigua de ***Armadillo*** ya que no incorpora ningún truco antidump, ni ***GetEnvironmentVariableA*** ni nada de nada..!!

Pues como no da para más el tema lo damos por terminado, podemos utilizar el programa de manera ilimitada y sin la presencia del ***Arma***, no hay referencias tipo ***unregistered*** ni nada, con todo esto damos por cumplidos los objetivos del tuto, espero que hayáis disfrutado como lo he hecho yo, un saludo para todo ***CracksLatinos*** y hasta la próxima….

Si alguien tiene alguna duda, pregunta, etc.., me puede escribir a mi correo electrónico en las siguientes direcciones:

syxe00@yahoo.es
syxe00@hotmail.com

Como siempre decir que todos los mensajes serán debidamente respondidos… chao!!